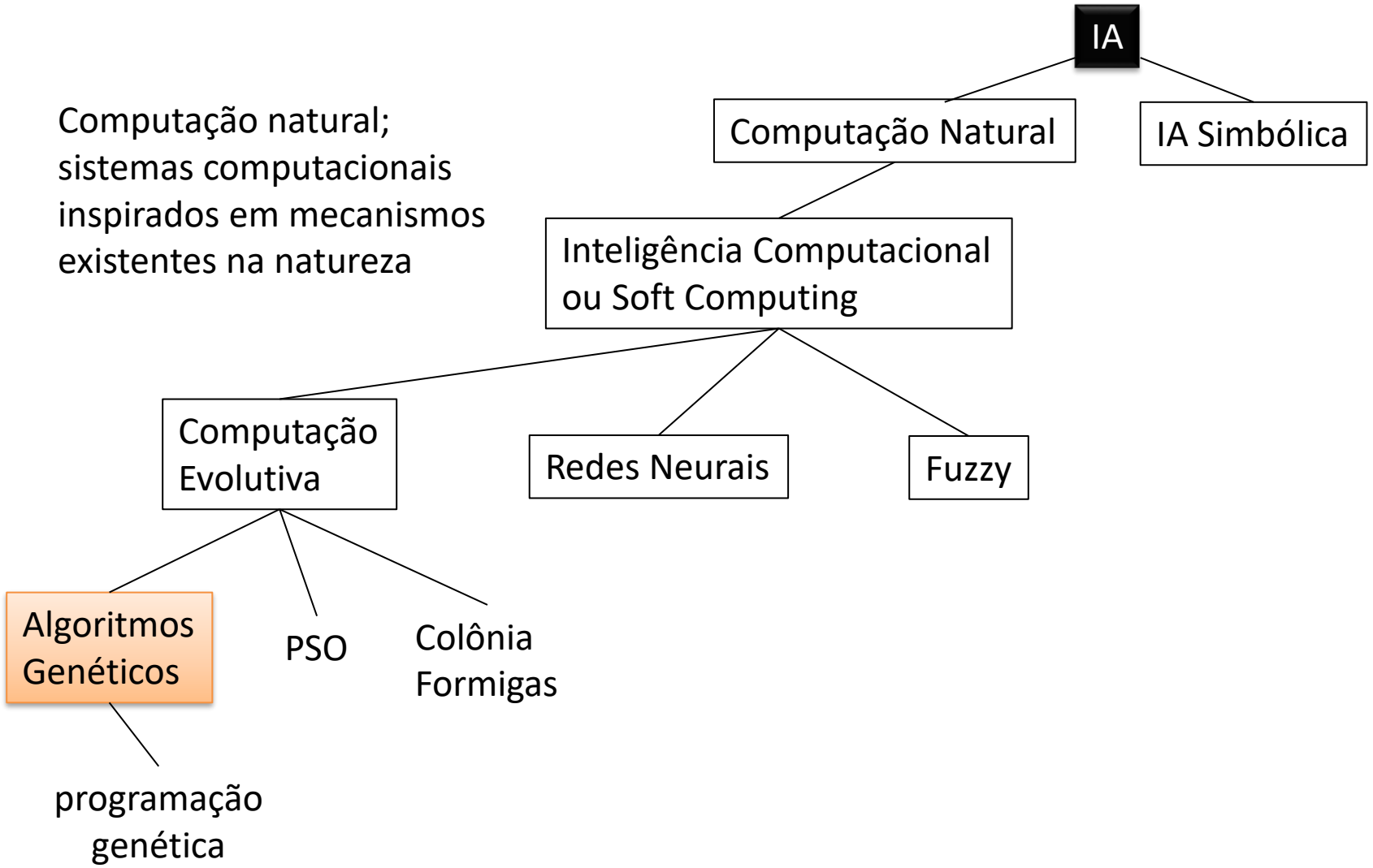
UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

ALGORITMOS GENÉTICOS

Metaheurísticas de Buscas

Computação Natural

Computação natural;
sistemas computacionais
inspirados em mecanismos
existentes na natureza



DEFINIÇÃO

"Algoritmos genéticos são métodos computacionais de **busca** baseados nos **mecanismos de evolução natural** e na **genética**. Em AGs, uma população de possíveis soluções evolui de acordo com **operadores probabilísticos** concebidos a partir de metáforas biológicas, de modo que há uma tendência de que, na média, os indivíduos representem **soluções cada vez melhores** à medida que o processo evolutivo continua." (Tanomaru, 1995)

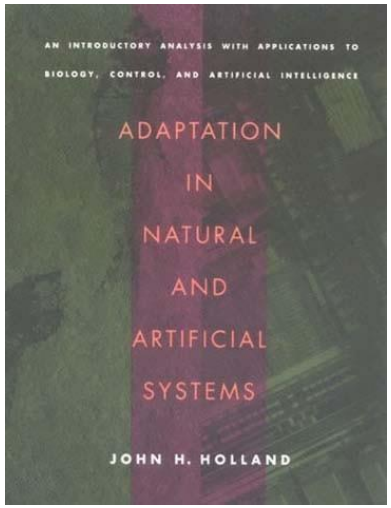
fonte: Tanomaru, J. *Motivação, Fundamentos e Aplicações de Algoritmos Genéticos, in: II Congresso Brasileiro de Redes Neurais, 1995.*

ALGORITMOS GENÉTICOS (AG)

Popularizados por **John Holland** podem ser considerados os primeiros modelos algorítmicos desenvolvidos inspirados nos sistemas genéticos.

Publicou em 1975 livro seminal de AG:

> *Holland, J. H. (1975).* ***Adaptation in natural and artificial systems****. Ann Arbor, MI: University of Michigan Press.*

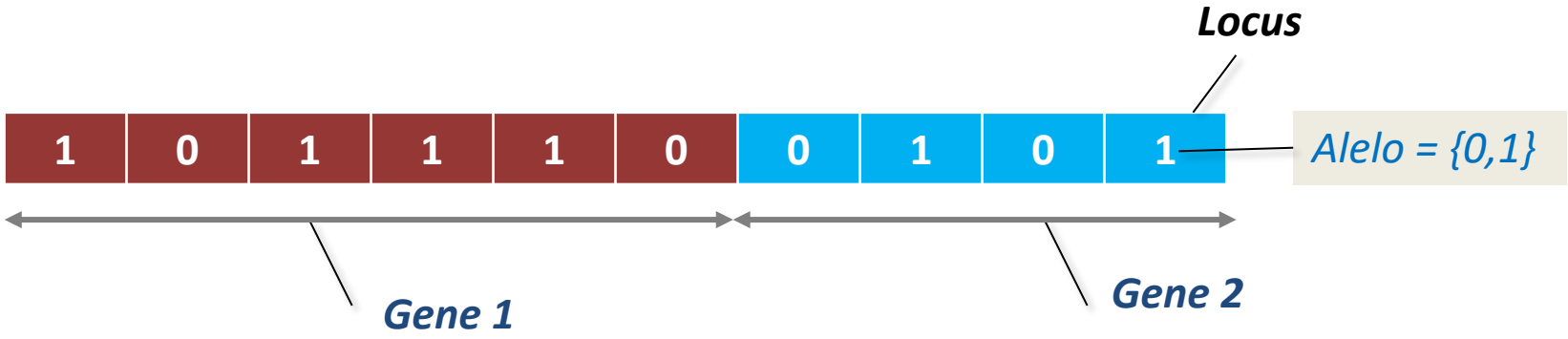


AG: Construtos básicos

- **CROMOSSOMO (ou indivíduo)**: **formado** por genes; normalmente representa uma solução completa a um problema.

- **GENE**: representa uma **faceta/característica** independente das demais.

- **ALELO**: é o **valor** armazenado em um locus.



Exemplo: PROBLEMA DAS 8 RAINHAS

Solução por Codificação binária (matriz binária) -> vetor de bits

0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	0	0	0	1	0	0
1	0	0	0	0	0	0	0
0	1	0	0	1	0	0	0
0	0	0	0	0	0	1	1

1ª coluna

2ª coluna



Gene representa uma coluna

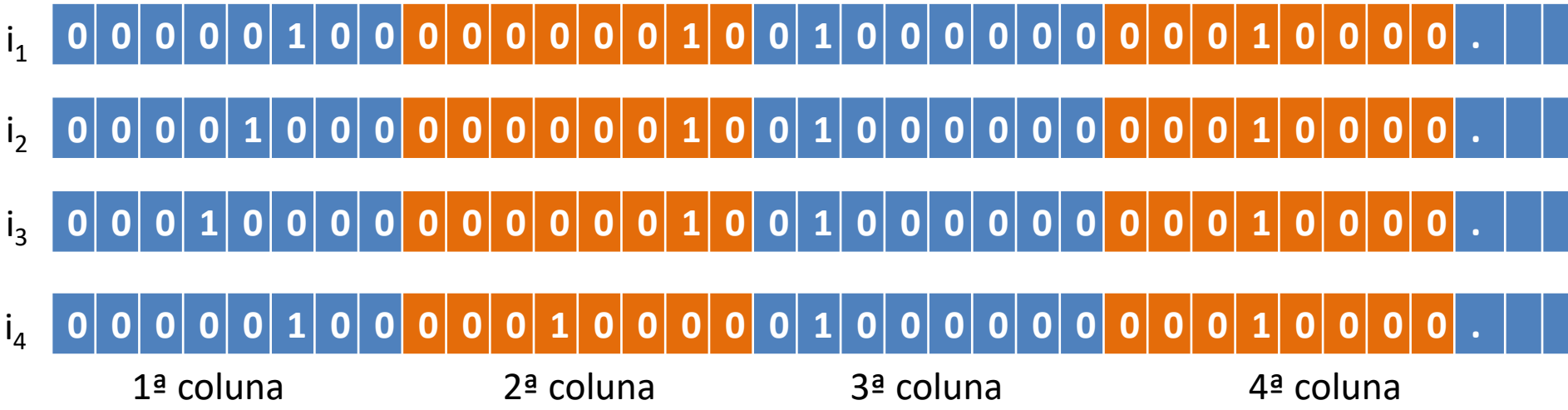
Cada **locus** representa uma posição da coluna *(locus gênico)*

Alelo: quando vale 1 indica que a posição está ocupada

AG: Construtos básicos

POPULAÇÃO: conjunto de cromossomos ou de indivíduos.

População de 4 indivíduos ou cromossomos para 8-Rainhas



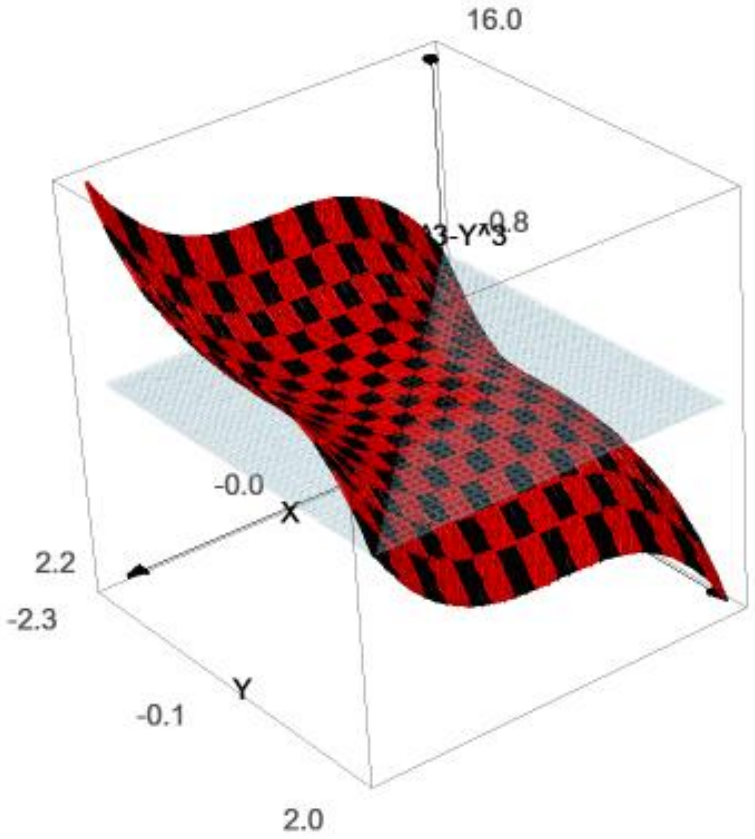
AG no contexto de Busca em IA

CARACTERÍSTICAS

- **Busca informada**: utiliza função de *fitness (avaliação)* que guia o algoritmo
- **Função de adequação ou fitness:** f(n) = h(n)
  f(indivíduo) = função de adequação ou fitness (indivíduo)
- **Busca local:** não guarda a informação do caminho para chegar à solução (mas os vizinhos não estão necessariamente próximos)
- **Estados sucessores** gerados pela combinação de dois estados pais
- **Útil quando** o espaço de estados é muito grande ou muito complexo para tratamento analítico

método de busca probabilístico de busca e otimização

EXERCÍCIO



Objetivo: maximizar $x^3 - y^3$

Tal que x e y ∈ [-2, 2] (no domínio dos inteiros)

DEFINA:

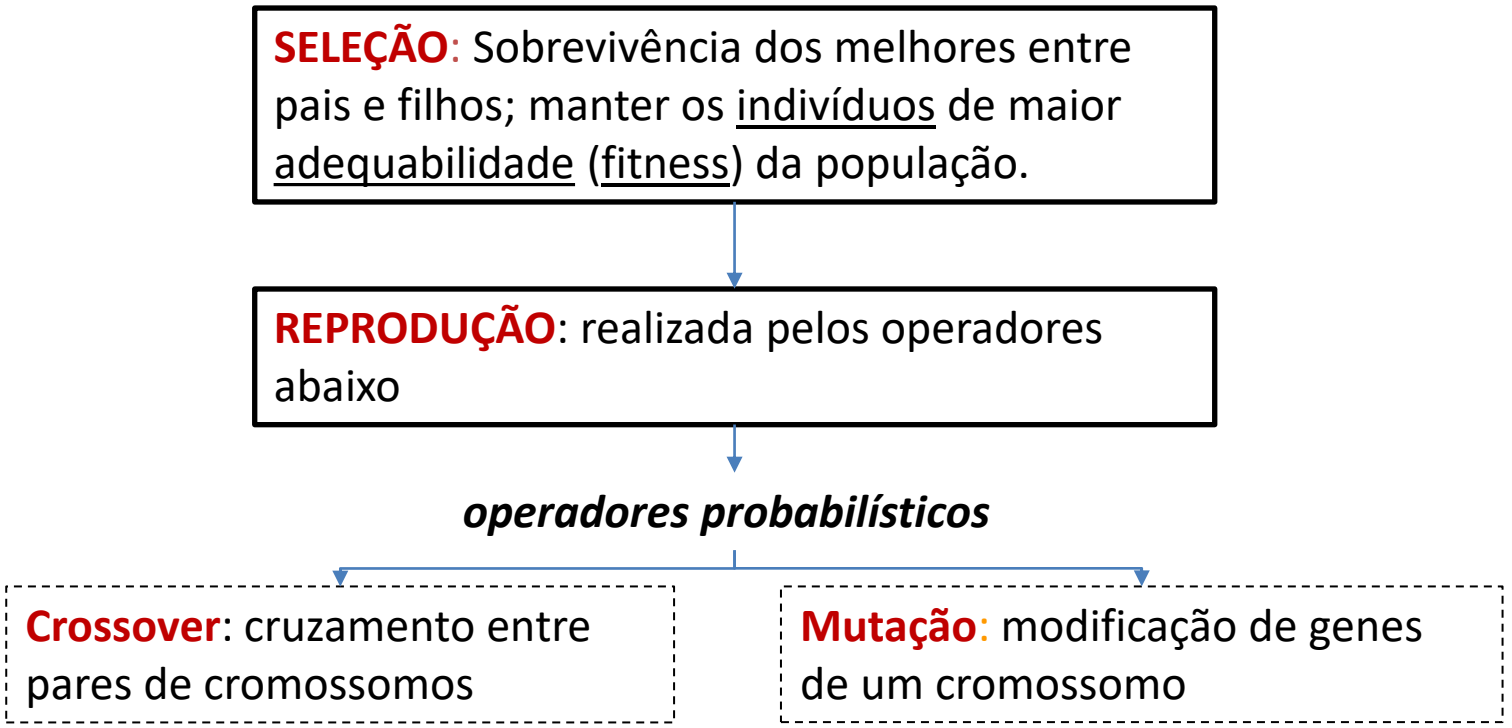
- Cromossomo ou indivíduo com seus genes/locus (codificação binária)

- fitness

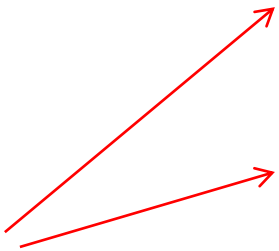
AG: aplicações

- Problemas de otimização (mín ou máx)
  - caixeiro viajante (min caminho percorrido)
- Problemas combinatoriais
  - bin packing (empacotamento)

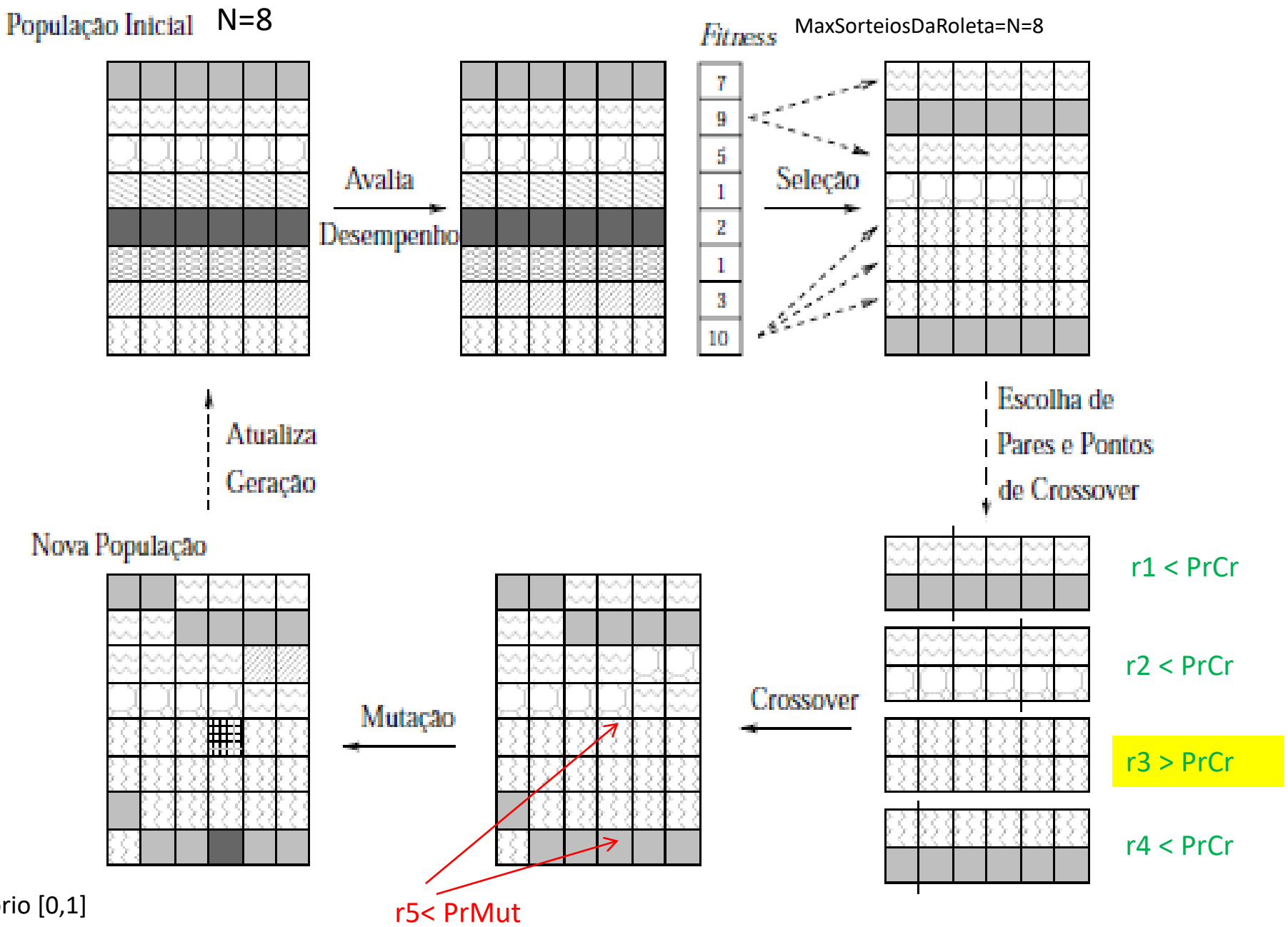
AG: etapas e operadores



ESQUEMA GERAL DE EVOLUÇÃO DE UM ALGORITMO GENÉTICO



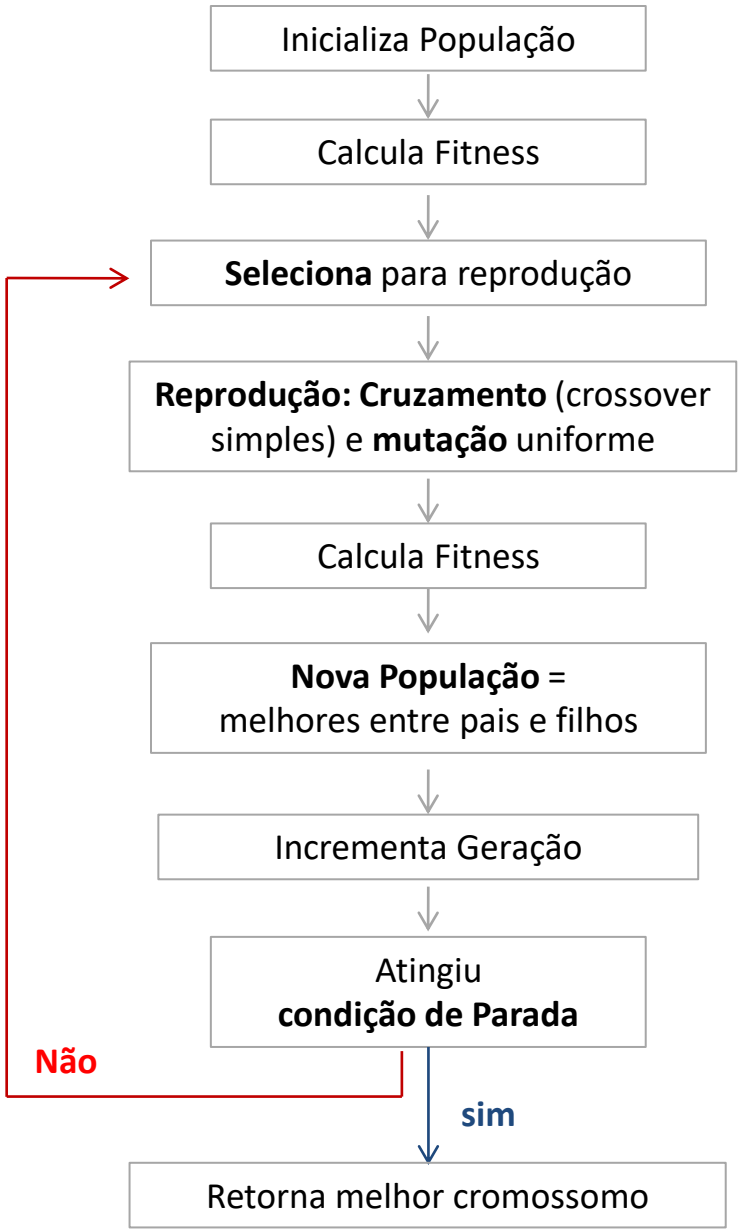
Esquema Geral de um Agoritmo Genético



leatório [0,1]

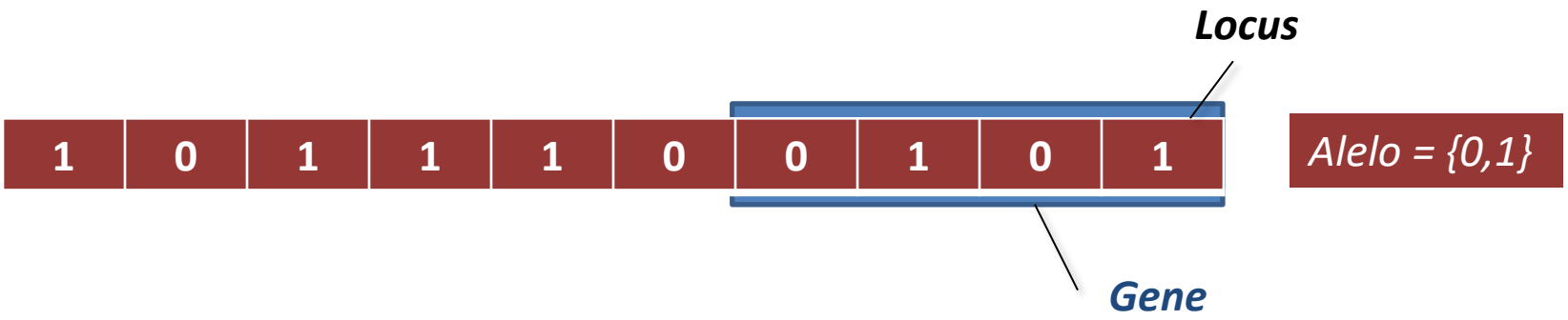
AG Canônico

- **Codificação:** Binária
- **Inicialização população**: aleatória
- **Seleção**
  - Reprodução (método da roleta)
- **Reprodução**
  - Crossover simples (pCROSS)
  - Mutação Uniforme (pMUT)
- **Sobrevivência**
  - Melhores (entre pais e filhos) irão compor a nova população
- **Condição de parada**
  - Geração = MaxGer ou
  - Fitness = máximo atingido ou
  - Estagnação do melhor fitness



AG Canônico: **Codificação binária**

Cromossomo no AG: cadeia de bits de tamanho L



Cada posição (locus) no cromossomo assume um dos dois possíveis alelos, 0 ou 1.

Exemplo: problema das N-Rainhas

AG Canônico: **seleção por roleta**

Método de seleção de cromossomos/indivíduos que seleciona K indivíduos (deve-se rodar k vezes a roleta).

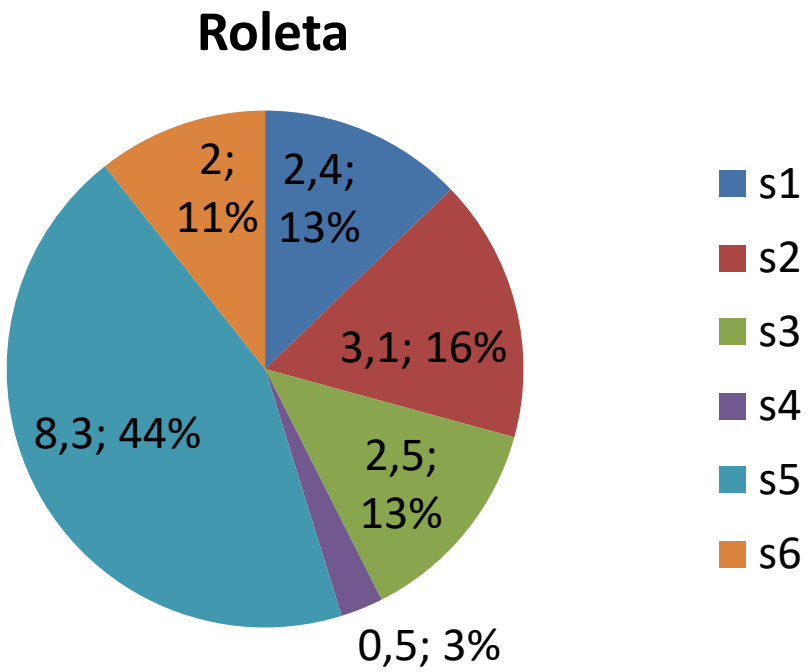
Os indivíduos de maior fitness tem maior probabilidade de serem selecionados para se reproduzirem (*método elitista*).

AG Canônico: **seleção por roleta**

1) Calcular a probabilidade de seleção de cada indivíduo $p(s_i)$ em função do seu fitness

para i=1,…,N: $$p(s_i) = \frac{fitness(s_i)}{\sum_{k=1}^{N} fitness(s_k)}$$

Indivíduo	Fitness	Norm.
S1	2,4	0,13
S2	3,1	0,16
S3	2,5	0,13
S4	0,5	0,03
S5	8,3	0,44
S6	2,0	0,11



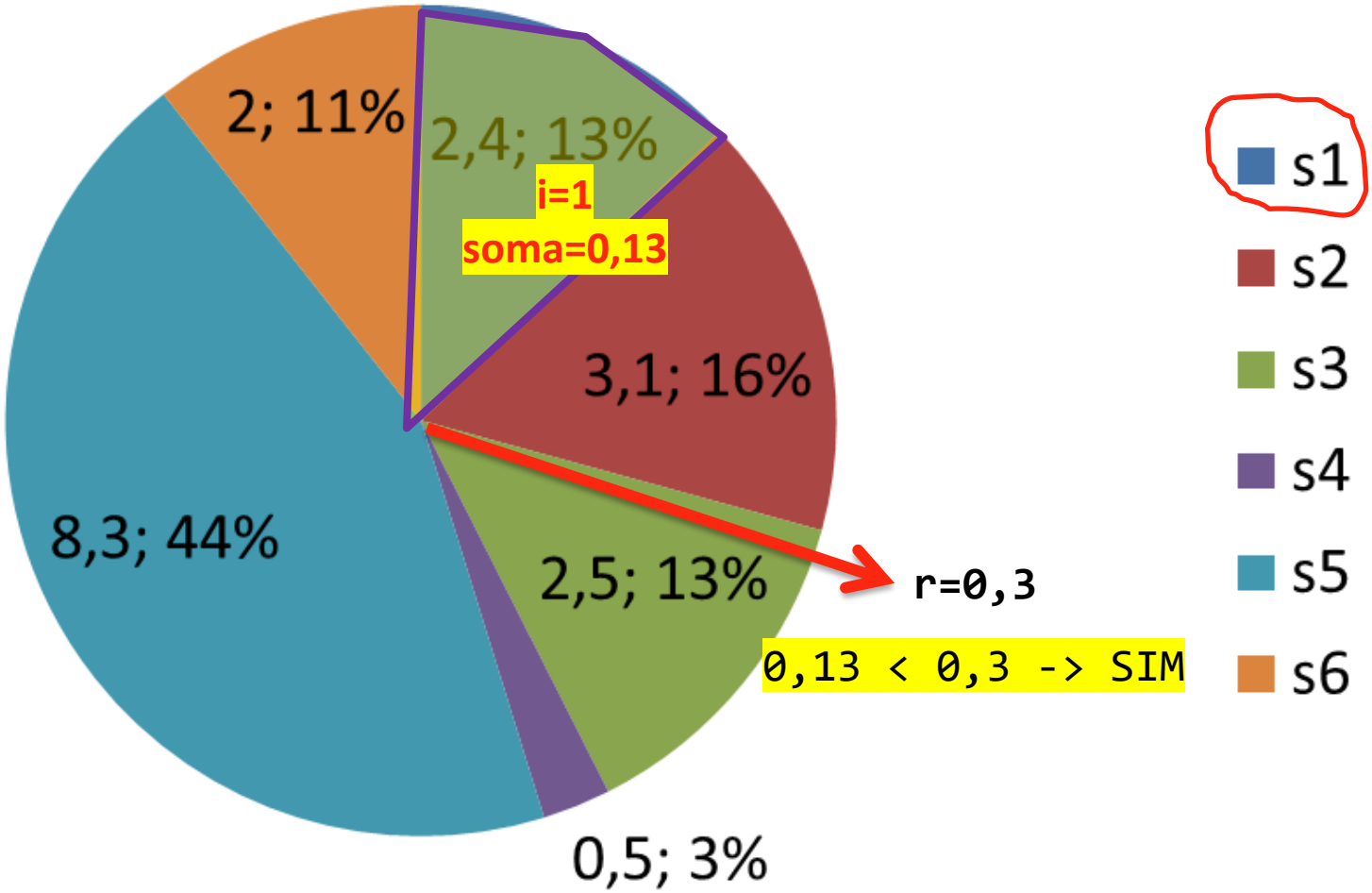
AG Canônico: **Seleção por roleta**

Algoritmo de seleção por roleta:
*sorteia **um indivíduo por chamada***

```
i=1;
soma = p(s_i)
Sorteia r ∈ [0, 1]
enquanto soma < r
    i=i + 1
    soma = soma + p(s_i)
fim enquanto
Retorna s_i
```

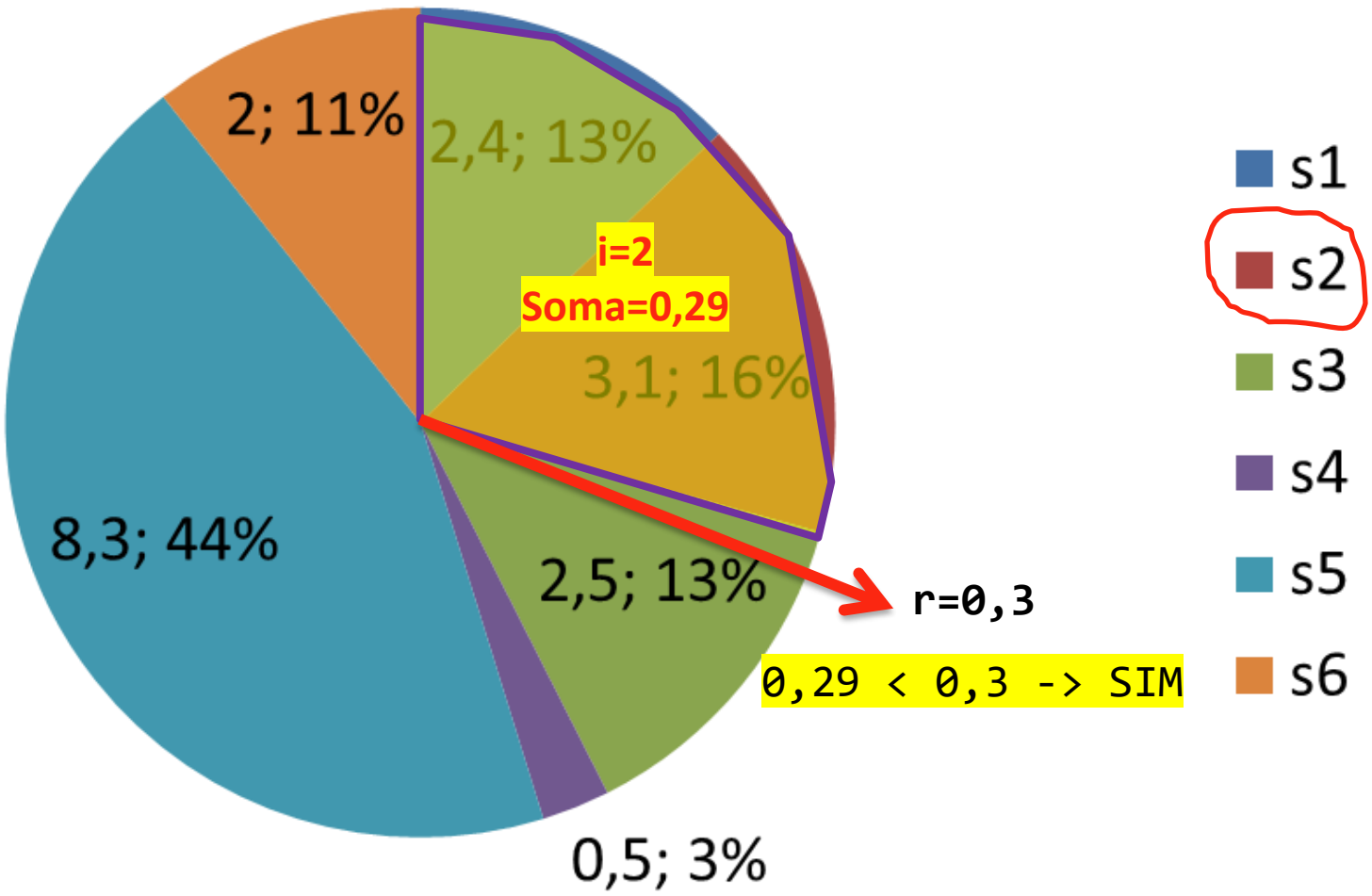
INICIALIZAÇÃO

```
i=1
soma := 0,13
r=0,3
```



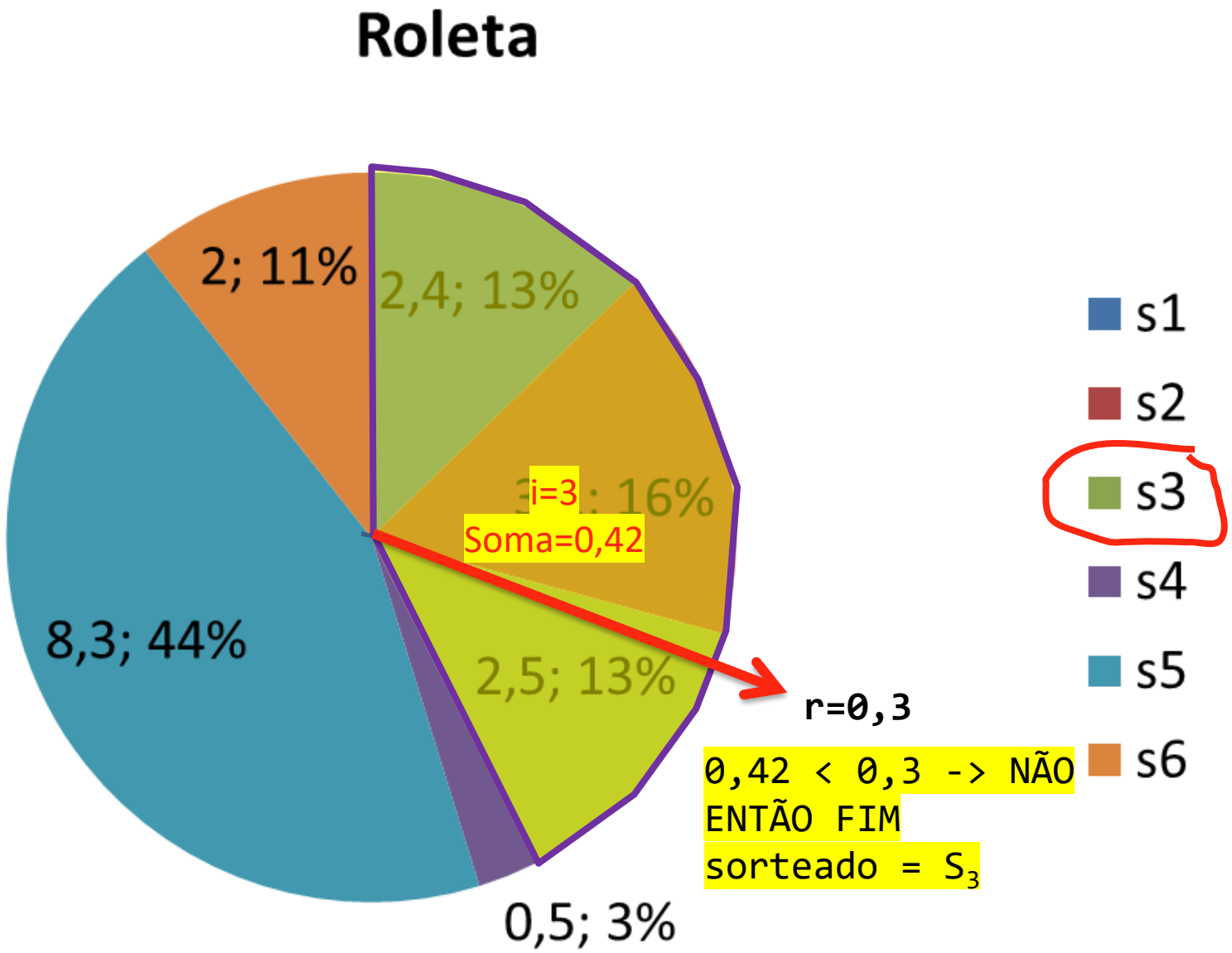
SORTEIO

i=2
Soma=0,29



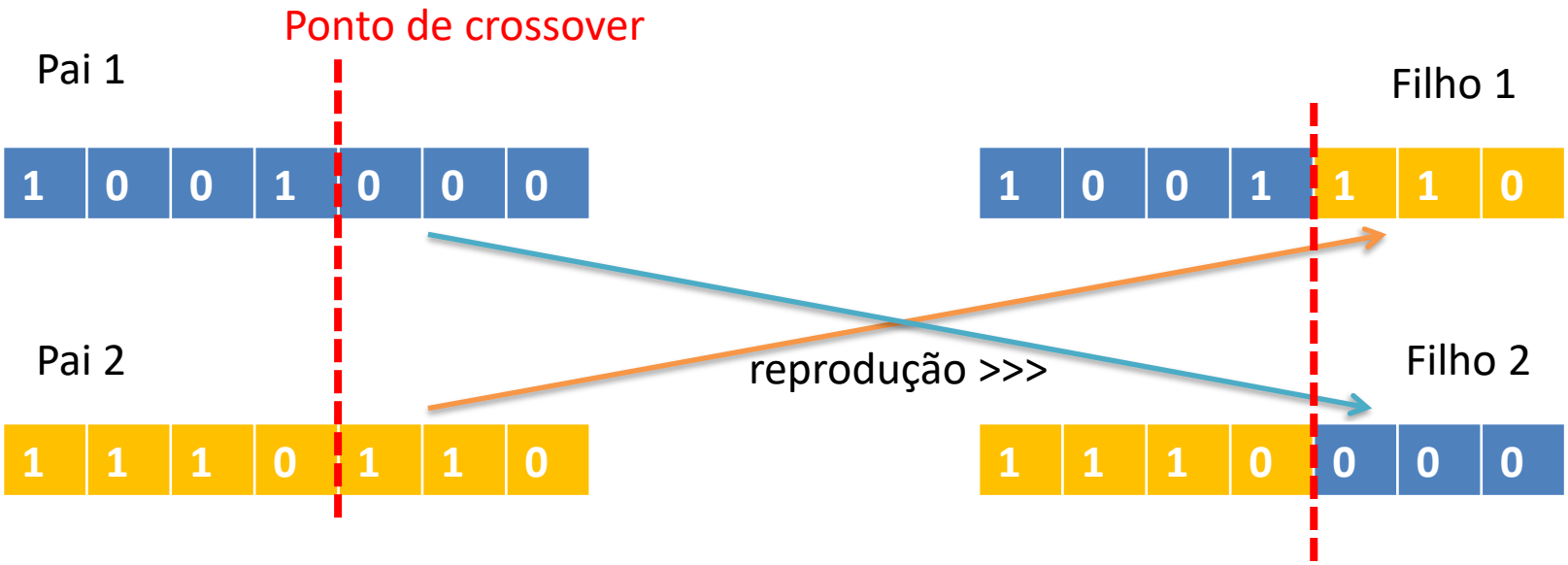
SORTEIO

i=3
Soma=0,42



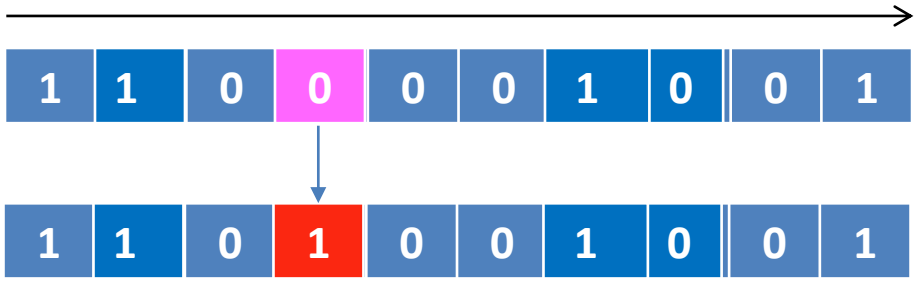
AG Canônico: **Reprodução e crossover**

Crossover Simples (1 ponto): posição de cruzamento escolhida aleatoriamente. Idêntica nos dois indivíduos que vão se reproduzir.



AG Canônico: **Reprodução e Mutação**

Dado um indivíduo, percorre locus a locus mutando ou não de acordo com a prob. de mutação



Ponto(s) de mutação com r <= pMut

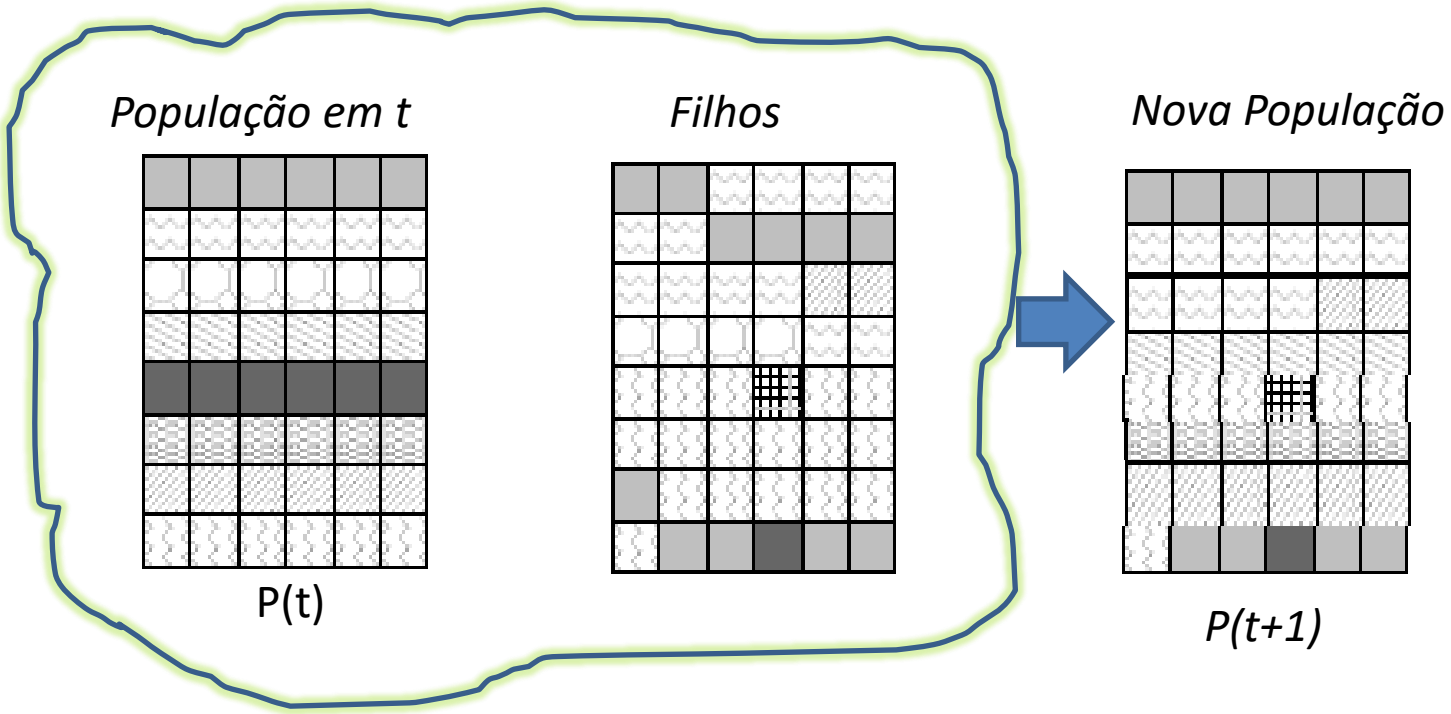
resultado da mutação

Posições são **escolhidas aleatoriamente**.

Mutação realizada com **probabilidade baixa** (tipicamente 0.05)

AG Canônico: **sobrevivência**

Sobrevivência dos Melhores: selecionar entre **pais e descendentes (filhos)** para compor a nova população



P(t+1)= Melhores(P(t) U Filhos)

AG Canônico: **pseudo-código**

```
C:={c1, ..., cn}   população inicial de tamanho N
R tamanho da descendência ou reprodução (normalmente R = N)
pCROSS: probabilidade de fazer crossover (valor típico 0.8)
pMUT: Probabilidade de fazer mutação nos alelos (valor típico 0.05)

AGCanônico(C, N, R, pCROSS, pMUT) {
   Para todo ci de C, calcular fitness(ci);
   geracao:=0;
   do {
      // D = Descendentes = nova geração, calculada a partir de C
      D := selecionar R cromossomos de C pelo método da roleta;
      D':= cruzamento(D); // geração de dois filhos por par (d1,d2),(d3,d4),...,(dk-1, dk)
          de D fazendo crossover com probabilidade pCROSS
      D":= para todo cromossomo di de D', para cada alelo aj de di, mutar aj
          com probabilidade pMUT;
      Para todo di de D", calcular fitness(di);
      // Selecionar melhores entre pais e filhos
      C := selecionar n melhores cromossomos de C ⊔ D";
      geracao++;
   } while (geracao<MAX_GERACOES and !objetivo-alcançado and
            !melhor-fitness estagnado);

   retornar ci de C com melhor fitness;
}
```

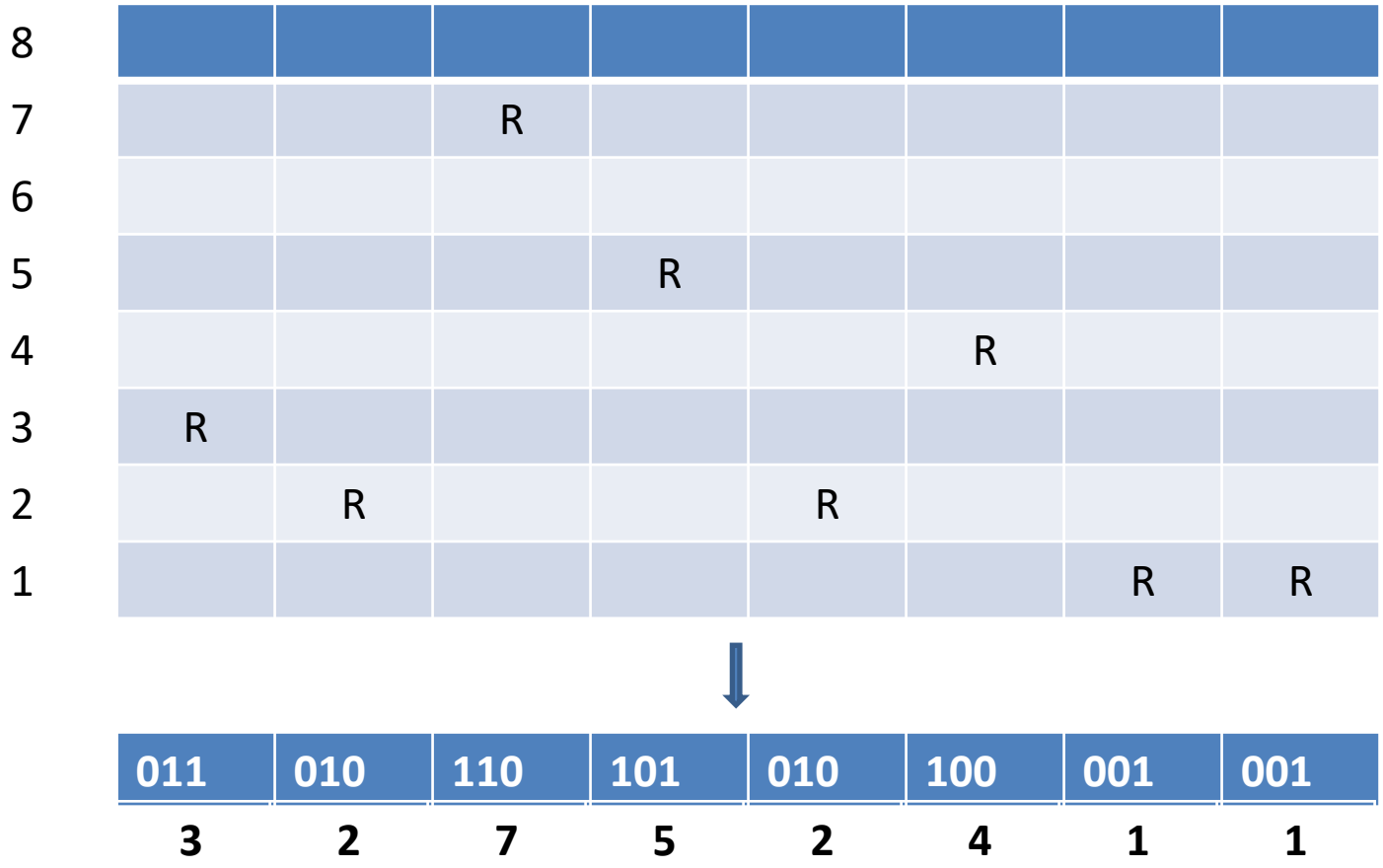
AG Canônico: resumo

- Representação do indivíduo: codificação binária
- **Seleção**
  - Roleta
- **Reprodução**
  - Crossover simples com seleção aleatória de pares (pais)
  - Mutação Uniforme
- **Sobrevivência**
  - Melhores entre pais e filhos para compor a nova população

CODIFICAÇÃO BINÁRIA X DECIMAL

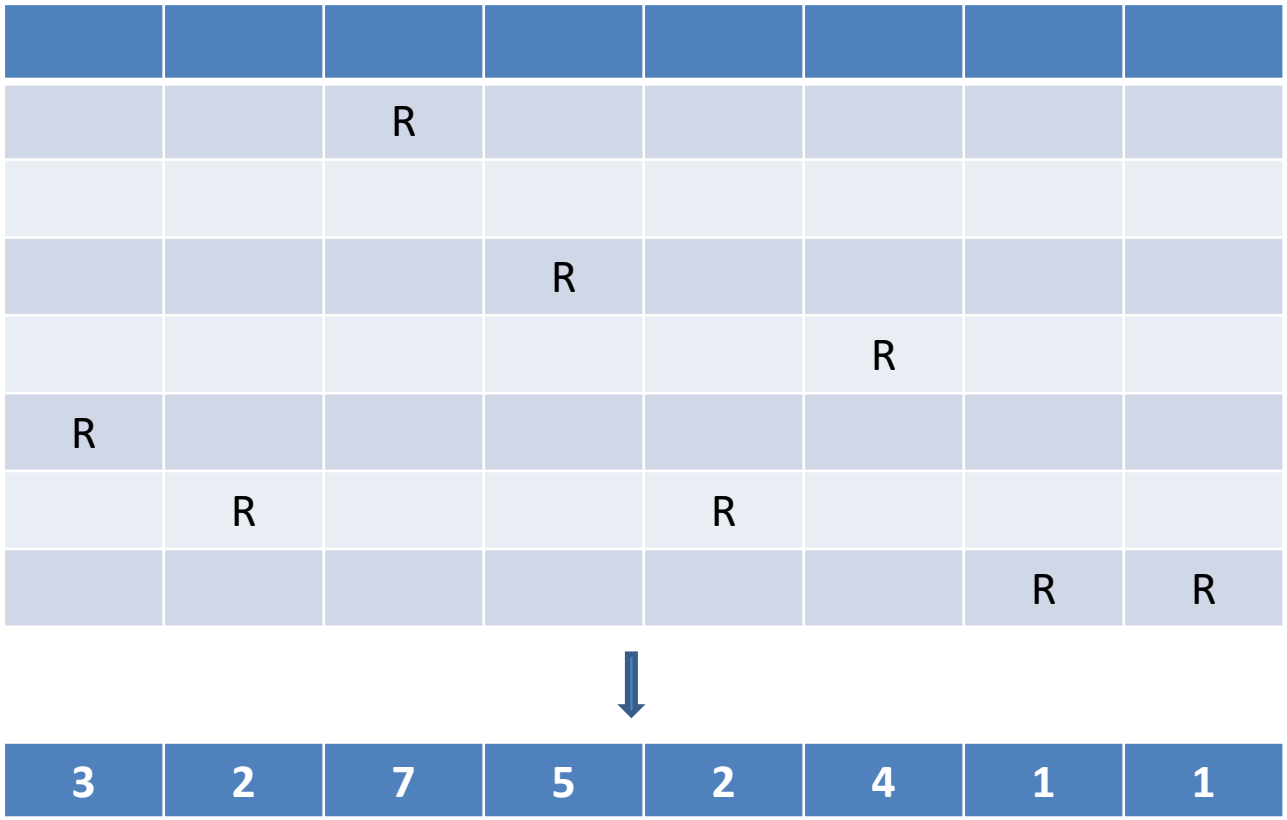
PROBLEMA DAS 8 RAINHAS

Solução por **codificação binária** (obtida por conversão inteiro-> binário)



PROBLEMA DAS 8 RAINHAS

Solução por **Codificação inteira**



Os operadores de crossover e mutação ainda funcionam?

Problema da Mochila

Problema da mochila: Um viajante deve levar consigo, apenas uma mochila. Essa mochila possui uma capacidade limitada e deve ser carregada apenas com objetos que serão úteis durante a viagem. Cada objeto é único e possui um peso e um determinado valor.

Que objetos devem ser levados pelo viajante de forma a maximizar o valor da mochila?

Problema da mochila

$$\text{maximizar } f(x) = \sum_{i=1}^{n} x_i v_i$$
$$\text{sujeito a } \sum_{i=1}^{n} x_i w_i \leq C, x_i \in \{0,1\} \quad (1)$$

onde v_i é o valor i-ésimo objeto, w_i o seu peso, x_i indica se o objeto aparece ou não na mochila e C define a capacidade da mochila (em termos de peso).

Obj(i=1,...n)	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Peso(kg)	3	8	12	2	8	4	4	5	1	1	8	6	4	3
Valor	1	3	1	8	9	3	2	8	5	1	1	6	3	2
Obj(i=1,...n)	15	16	17	18	19	20	21	22	23	24	25	26	27	28
Peso(kg)	3	5	7	3	5	7	4	3	7	2	3	5	4	3
Valor	5	2	3	8	9	3	2	4	5	4	3	1	3	2

Capacidade da mochila=28Kg

Problema da mochila

Solução por Codificação binária (matriz binária) -> vetor de bits

- item presente: 1
- item ausente: 0

ITENS

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
0	0	1	0	1	1	0	1	0	0	1	1	1	0	1	0	1	0	0	0	0	1	1	1	1	0	0	0

AG EM PROBLEMAS DE PERMUTAÇÃO

Problemas de permutação

- **Solução** = determina a ordem de uma sequência de eventos

- **Exemplos**
  - **caixeiro viajante***: determinar a ordem de visita das cidades minimizando a distância percorrida (*problema de otimização de rotas)
  - **scheduling:** *ordem de operações para minimizar tempo de produção de peças)*

- *Que codificação utilizar?*
- *Quais operadores utilizar?*

Problemas de Permutação

Normalmente utiliza-se

Codificação da ordem: gene indica a ordem de visitação de cada uma das cidades de A a I

`gene=ordem de visita  alelo=[1, número de cidades]`

Caixeiro viajante

cidade	A	B	C	D	E	F	G	H	I	= fixas
ordem visita	**5**	**2**	**9**	**4**	**7**	**6**	**1**	**3**	**8**	= **cromossomo**

Codificação posicional: gene indica a cidade que será visitada em 1º., 2º., ….

`gene=id da cidade  alelo={ids das cidades}`

Caixeiro viajante

ordem visita	1	2	3	4	5	6	7	8	9	= fixa
cidade	**G**	**B**	**I**	**D**	**A**	**F**	**E**	**I**	**C**	= **cromossomo**

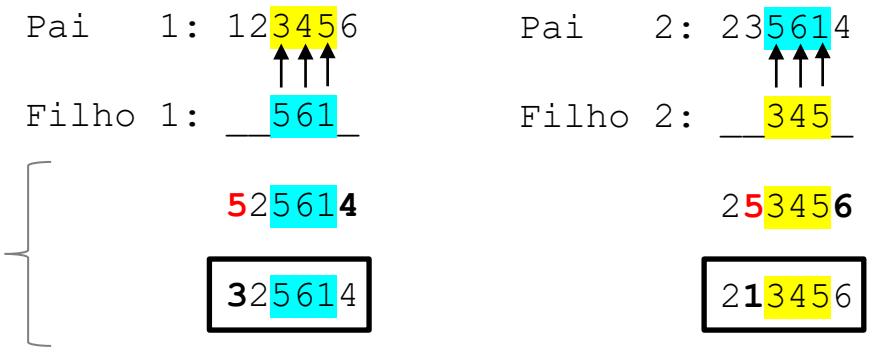
AG em Problemas de Permutação: tipos de crossover

Crossover de Mapeamento Parcial (**PMX**):
não gera infactibilidade

1. sorteio das posições i e j

2. Gerar filhos: troca-se material genético entre os pais e cria-se um mapeamento na direção **do filho → pai**

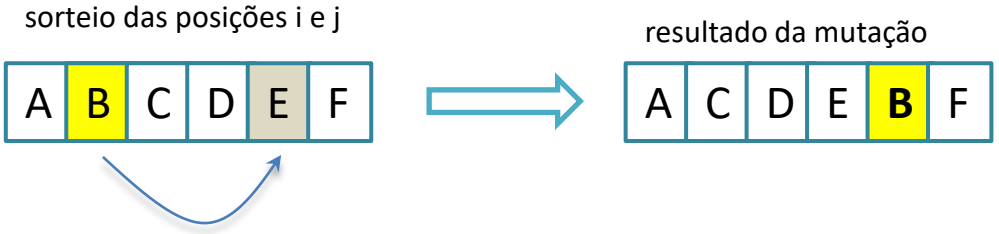
3. São feitos **mapeamentos sucessivos** para preencher as posições faltantes e eliminar repetições sempre na direção filho → pai



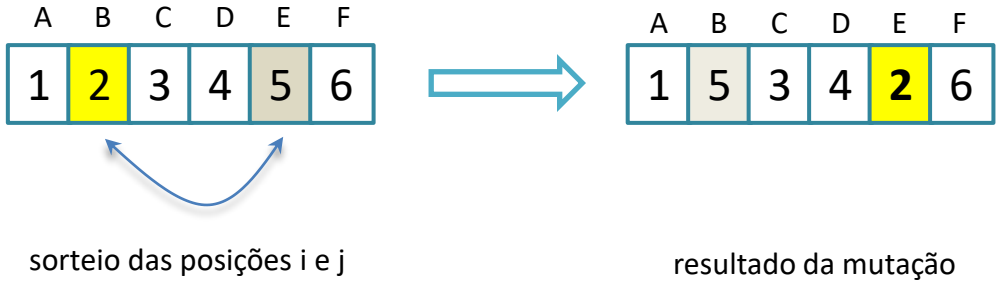
Em vermelho: repetições
*Em **negrito**: substituições*

AG em Problemas de Permutação: tipos de mutação

Mutação baseada em posição:
retira locus da posição i e insere em j



Mutação baseada em ordem:
troca alelo (valor) da posição i com o da j



AG em Problemas de Permutação: tipos de mutação

Mutação por embaralhamento

Selecionar aleatoriamente uma sublista e embaralhá-la.



sorteio das posições i=inicial e j=final

resultado da mutação